Course Design Report
High-Performance Gaming Traffic Prioritizer

Author: Team 2 Bohan Jin(3131953J), Zeyu Lei(3119470L), Zhi Qi(3131933Q), Jiandong Yang(3131937Y), Chuang Zong(3130036Z).

1. Introduction

1.1 Background and Motivation

Reliable network communication is essential for customers' online gaming experience. However, in the shared local network, bandwidth-intensive activities (i.e. video streaming and file transfers) can increase network congestion, which leads to packet-queuing delays for online gaming. Conventional consumer routers generally provide basic Quality of Service (QoS) mechanisms, but these often require manual configuration and may not respond quickly enough to rapidly changing traffic conditions. This project therefore develops a High-Performance Gaming Traffic Prioritizer: with helps of Raspberry Pi 4B and modern C++23, this project implemented a transparent network appliance, which is to prioritise gaming network over non-critical high-bandwidth flows.

1.2 Project Objectives

The primary objective is to develop a real-time traffic management system capable of processing network packets with minimal latency. The project aims

- To classify network traffic dynamically;
- To implement a thread-safe architecture using efficient data structures and event-driven processing;
- To minimise unnecessary data copying and processing overhead between user space and the Linux networking stack;
- To apply traffic prioritisation and bandwidth control automatically.

The overall objective is to demonstrate how real-time programming techniques can be applied to a practical networking problem while maintaining reliability, maintainability and reproducibility.

2 System Design: coding structure

2.1 System Architecture

The software architecture of the High-Performance Gaming Traffic Prioritizer is organised into four main layers (figure 1): the application layer, layer of control plane, layer of data plane, and layer of network service. This 4-layer architecture separates system management into high-frequency packet processing, improving maintainability and reducing unnecessary dependencies between components.

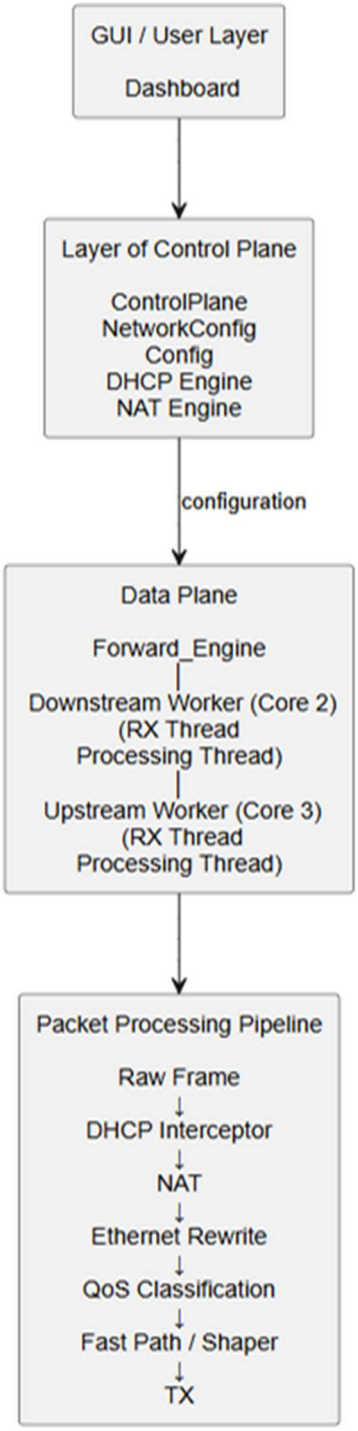


Figure1: System Architecture (source: plotted by author by Plantmul code https://plantuml.com/)

The application layer is represented by the App class, which acts as the main system coordinator. It is responsible for initialising, configuring, starting and stopping the major system components. The layer of control plane manages configuration and system-level services through components such as ControlPlane, NetworkConfig, NatEngine, DhcpEngine, and the Qt6 graphical interface. These components handle configuration changes and network services without directly implementing the main packet-processing pipeline.

The layer of data plane is centred on Forward_Engine. It manages the WAN-to-LAN and LAN-to-WAN forwarding paths and creates the corresponding worker threads. Within each forwarding path, PacketConsumer provides the packet-processing pipeline, including packet parsing, NAT processing, Ethernet header modification, traffic classification and QoS routing.

The layer of Packet Processing contains specialised components such as NatEngine, DhcpEngine and Shaper. Each component provides a clearly defined service to the data plane rather than combining all networking functions into a single class. This separation of responsibilities allows individual components to be modified independently while maintaining a clear overall system structure.

2.2 Object-Oriented Design

The system uses an object-oriented design to encapsulate networking functionality into specialised classes. The main design principle is separation of concerns, where each class has a clearly defined responsibility.

The App class functions as the application-level coordinator (see in figure 2). It owns and initialises major system components, including the NAT and DHCP engines, forwarding engine, shapers and network configuration services. It therefore manages component lifecycles without implementing packet-processing logic itself.

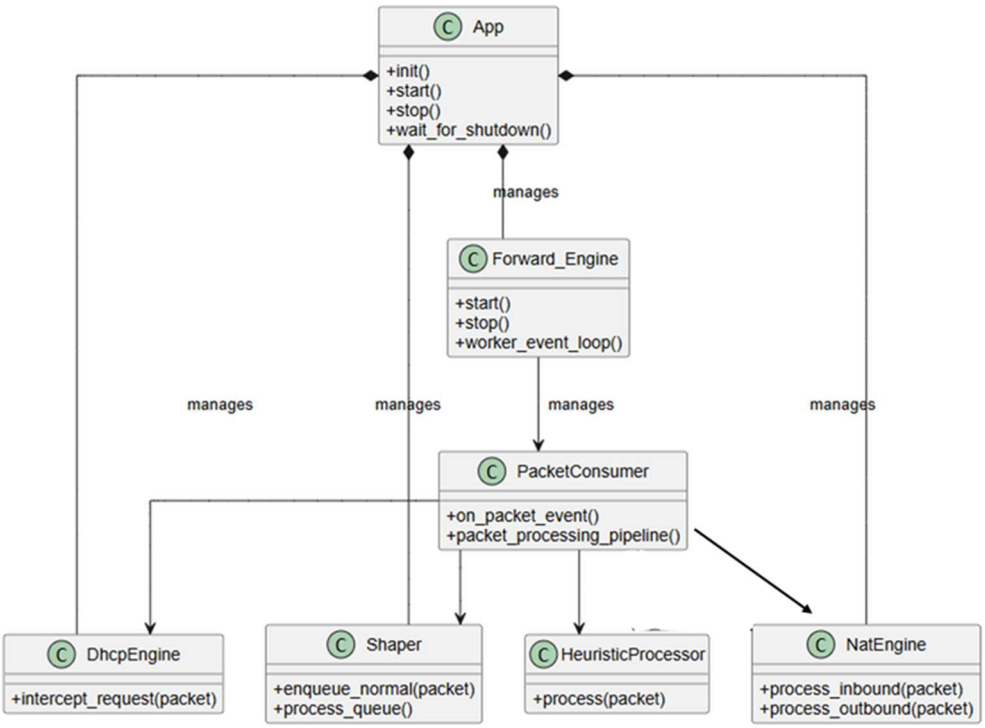


Figure 2: UML for system (source: plotted by author by Plantmul code https://plantuml.com/)

Forward_Engine is responsible for the data-plane forwarding infrastructure. It manages the WAN and LAN interfaces, worker threads and the resources required for packet forwarding. The actual packet-processing logic is encapsulated within PacketConsumer. This prevents the forwarding engine from becoming tightly coupled to individual packet-processing operations. PacketConsumer implements

the ordered packet-processing pipeline. It coordinates DHCP interception, NAT processing, Ethernet header rewriting, traffic classification and QoS routing. Its pipeline-based structure allows individual processing stages to remain logically separated while ensuring that packets follow a deterministic processing sequence. HeuristicProcessor encapsulates traffic classification. It receives a parsed packet and determines its priority without requiring PacketConsumer to implement the classification algorithm directly. Similarly, Shaper encapsulates bandwidth-limiting and packet-queue management, separating traffic control from packet classification. NatEngine and DhcpEngine provide independent network services for address translation and DHCP processing. They are managed at the application level and supplied to the data-plane components as dependencies, avoiding duplicated ownership.

Overall, the design uses encapsulation and clear component ownership to reduce coupling between modules. This structure improves code readability, which meanwhile supports long-term maintainability and reliability of my App (seeing details in appendix).

3 Code Implementation

3.1 implementation of Real-Time Thread

The High-Performance Gaming Traffic Prioritizer adopts a multi-threaded architecture to support concurrent and low-latency packet processing. The Forward_Engine creates two independent forwarding paths: a downstream path on Core 2 for WAN-to-LAN traffic and an upstream path on Core 3 for LAN-to-WAN traffic. Each path contains an RX thread and a processing thread.

The RX thread is responsible for receiving Ethernet frames from the corresponding raw socket. It waits for network events using a blocking poll() call, which is to avoid the continuously checking of interface. When a frame becomes available, the thread copies it into the fixed-size RxFrameCopy structure and inserts it into the worker's SpscRingBuffer<RxFrameCopy, 1024>. The processing thread consumes frames from the same queue, and it then passes each frame to PacketConsumer. The packet is processed through the defined pipeline, including DHCP interception, NAT processing and so on. High-priority traffic is forwarded through the fast path, while normal traffic is passed to the Shaper for controlled transmission to the other Ethernet interface.

This design prevents a processing operation from directly blocking Ethernet frame reception and it can allow the two stages to operate concurrently.

3.3 Event-Driven Processing

Communication between the RX and processing threads combines an SPSC ring buffer with eventfd notifications. The RX thread acts as the producer. It inserts frames using push() to copy packet into memory (named as frame_q). Then, the processing thread acts as the consumer using pop() to get the packet out of the memory. This provides a simple and controlled producer-consumer relationship for transferring frames between the two threads.

After successfully placing a frame into the queue, the RX thread calls eventfd_write() to notify the processing thread. The processing thread waits on the frame-event descriptor using

a blocking poll() call. This thread is therefore awakened only when new work is available. This avoids a continuously running polling loop and it so reduces unnecessary CPU consumption.

The architecture also uses callback-based event notification for packet statistics and transmission results. CallbackRegistry dispatches packet & batch events to registered observers, while the transmission callback reports the result of the Shaper transmission. Consequently, monitoring and reporting functions remain separated from the main packet-processing logic.

The complete event-driven path can therefore be summarised as:

Ethernet Frame → RX Thread → SPSC Ring Buffer → eventfd → Processing Thread → PacketConsumer → QoS Decision → Fast Path / Shaper → Ethernet Transmission

This architecture combines concurrent threads, blocking event notification, lock-free producer-consumer communication and callbacks to provide an efficient real-time packet-processing system.

Remark:

- how to run: we already added the readme.md in our github release. In brief, you can go from the project root, run:
  chmod +x ./start_release.sh
  sudo ./start_release.sh
  The script will automatically check dependencies, build the release version, and launch the GUI on the DSI display. You can also run the program from the repository root, while using sudo, as the data plane requires raw socket and network interface access.
- Social media poster: The project was also advertised through social media using a promotional description. The post communicated the practical problem addressed by the project to manage latency-sensitive gaming traffic impacted by the bandwidth-intensive traffic. It also highlighted the final evaluation results. (*http://xhslink.com/o/4Y6GghFouzZ*).

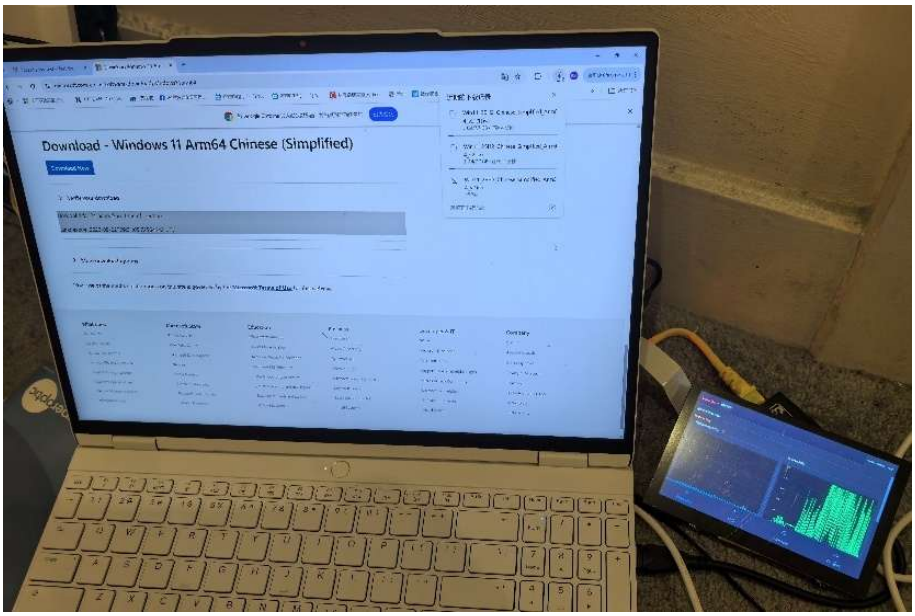
4. Testing and Evaluation

The High-Performance Gaming Traffic Prioritizer was evaluated using three network traffic scenarios to assess the effectiveness of the QoS mechanism. The tests focused on large packet traffic, small packet traffic and a combination of both traffic types.

4.1 Large-Packet Traffic Test

The first test evaluated the behaviour of the system under bandwidth-intensive traffic. A downloading operation of Windows 11 was used to generate large network transfers (packet source: https://www.microsoft.com/zh-cn/software-download/windows11). Without bandwidth limiting, the download reached a peak throughput of approximately 900 Mbps, with an average throughput of approximately 300 Mbps (figure 3b). When the QoS bandwidth limit was enabled, the throughput speed immediately decreased with an average of approximately 30 Mbps.

This demonstrates that the Shaper can respond to effectively restrict high-bandwidth traffic. The immediate reduction in throughput also indicates that the QoS configuration is applied during active traffic.



(a)

(b)

(c)

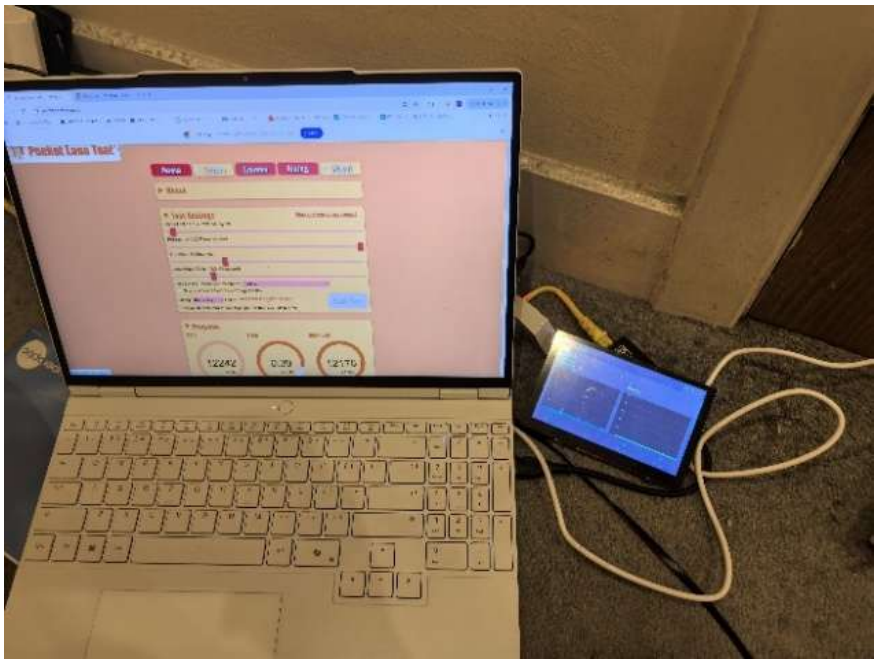
Figure 3: Testing for large Network Packet: a) experimental setting for downloading big packets b) network speed without traffic control on right screen, c) network speed after traffic control on right screen

4.2 Small-Packet Traffic Test

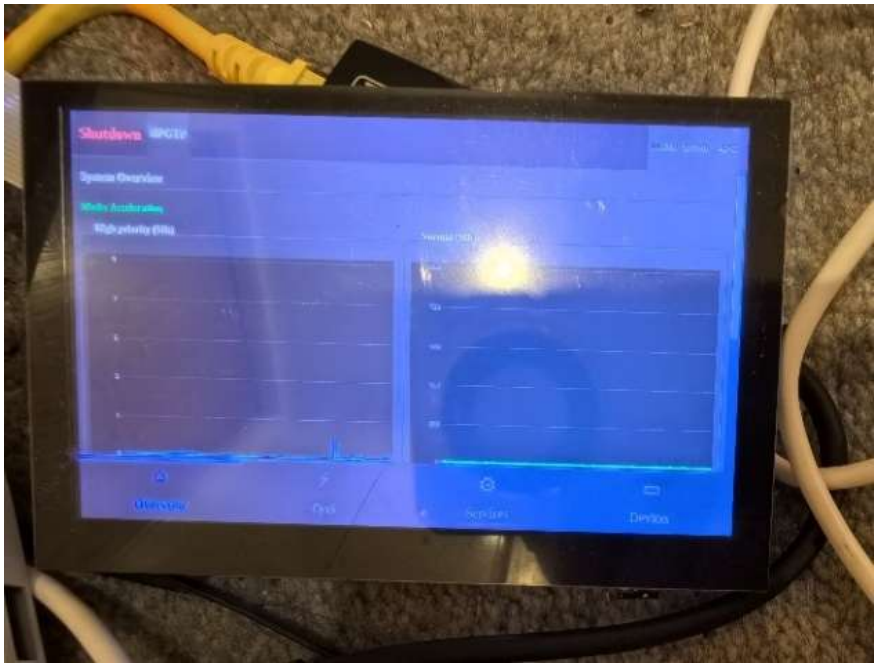
The second test examined the behaviour of small network packets. A packet-loss testing website was configured to generate packets with a size of approximately 200 bytes (figure

4a). This is to represent the type of frequent small packets. (*Packet source: https://packetlosstest.com/)*

Without bandwidth limiting, the measured throughput reached approximately 1 Mbps. After enabling the bandwidth limitation, the measured throughput remained approximately 1 Mbps (figure 4b). This result showed that the small-packet traffic was not significantly reduced by the bandwidth-control mechanism.



(a)



(b)

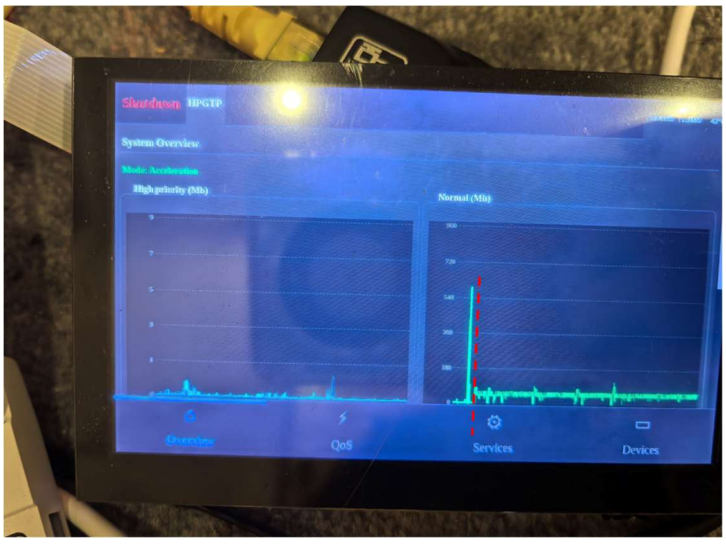
Figure 4: Testing for small Network Packet: a) experimental setting for downloading small packets b) network speed after traffic control on left screen.

4.3 Combined Large- and Small-Packet Traffic Test

The third test evaluated the main QoS objective by generating large and small packet traffic simultaneously. Under unrestricted conditions, the large traffic reached a peak of approximately 900 Mbps, with an average of approximately 300 Mbps (figure 5a right screen). After the bandwidth limit was enabled, the large traffic immediately decreased to approximately 95 Mbps, with an average of approximately 30 Mbps (figure 5a right screen). At the same time, the throughput of the small-packet traffic increased from approximately 1 Mbps to 2 Mbps (figure 5a left screen). This indicates that limiting the bandwidth-intensive flow released network capacity for the higher-priority small-packet traffic.

The combined test therefore provides the strong evidence for the effectiveness of my prioritisation mechanism. The system does not simply reduce total network throughput; instead, it actively controls bandwidth-intensive traffic while allowing higher-priority traffic to continue to get the network capacity.

(a)



(b)

Figure 5: Comparison of network for large- and small-packet traffic: a) before network control; b) after network control

5 Team Member Contributions

MSE981 (Bohan Jin) – Main Coding Organiser & Team Leader: his key role is to lead the overall software development, who also coordinated the team's coding activities. In details, he took primary responsibility for organising the codebase, while he was coordinating development tasks, and ensuring consistency across different modules.

TommyJD (Jiandong Yang) – Application Evaluation, Testing, Debugging & Architecture Design: he led the evaluation & testing of the application. In his work, he designed the process of packet download for testing network speed. He also worked across different components to ensure that the application remained stable with the project's architectural requirements.

Jayson030 (Zeyu Lei) – Data Pipeline & Frontend/Backend Integration: He was allocated to be responsible for developing the data pipeline connecting the application with the dashboard components. He worked on integrating the frontend and backend to ensure reliable data transmission. Meanwhile, he made some contributions to the project's social media activities, including supporting the communication and promotion of the project through relevant social media channels.

chuangzong1023 (Chuang Zong) – Dashboard Data Visualisation & Performance Monitoring: he was closely working with *Jayson030 (Zeyu Lei)* for dashboard function. He worked on development of the dashboard, who focused on presenting system information through data visualisations. He also implemented dashboard components for monitoring system performance and operational status.

wudidezhi-debug (Zhi Qi) – Marketing Leader & DHCP/NAT Engine Developer: his key work is to lead the project's marketing-related activities, who coordinated with *Jayson030 (Zeyu Lei)* for the promotion of the project. In parallel, he contributed to the development of the networking layer, with a particular focus on the DHCP and NAT engines.

6. Summary

This project developed a real-time High-Performance Gaming Traffic Prioritizer to address the effect of bandwidth-intensive network traffic on latency-sensitive communication. The implementation combines an object-oriented software architecture with event-driven real-time programming techniques.

Real-time packet processing is implemented using RX and processing threads. The SPSC ring buffer provides controlled communication between the threads, while eventfd and blocking poll() calls provide event-driven thread wake-up. This avoids unnecessary busy polling and it can allow processing resources to be concentrated on actual network events.

The evaluation demonstrated that the QoS mechanism can substantially restrict bandwidth-intensive traffic while improving the network traffic to small packets. Overall, the project demonstrates how object-oriented design programming and event-driven real-time techniques can be combined to implement a practical low-latency networking system.

Appendix Detailed UML structure

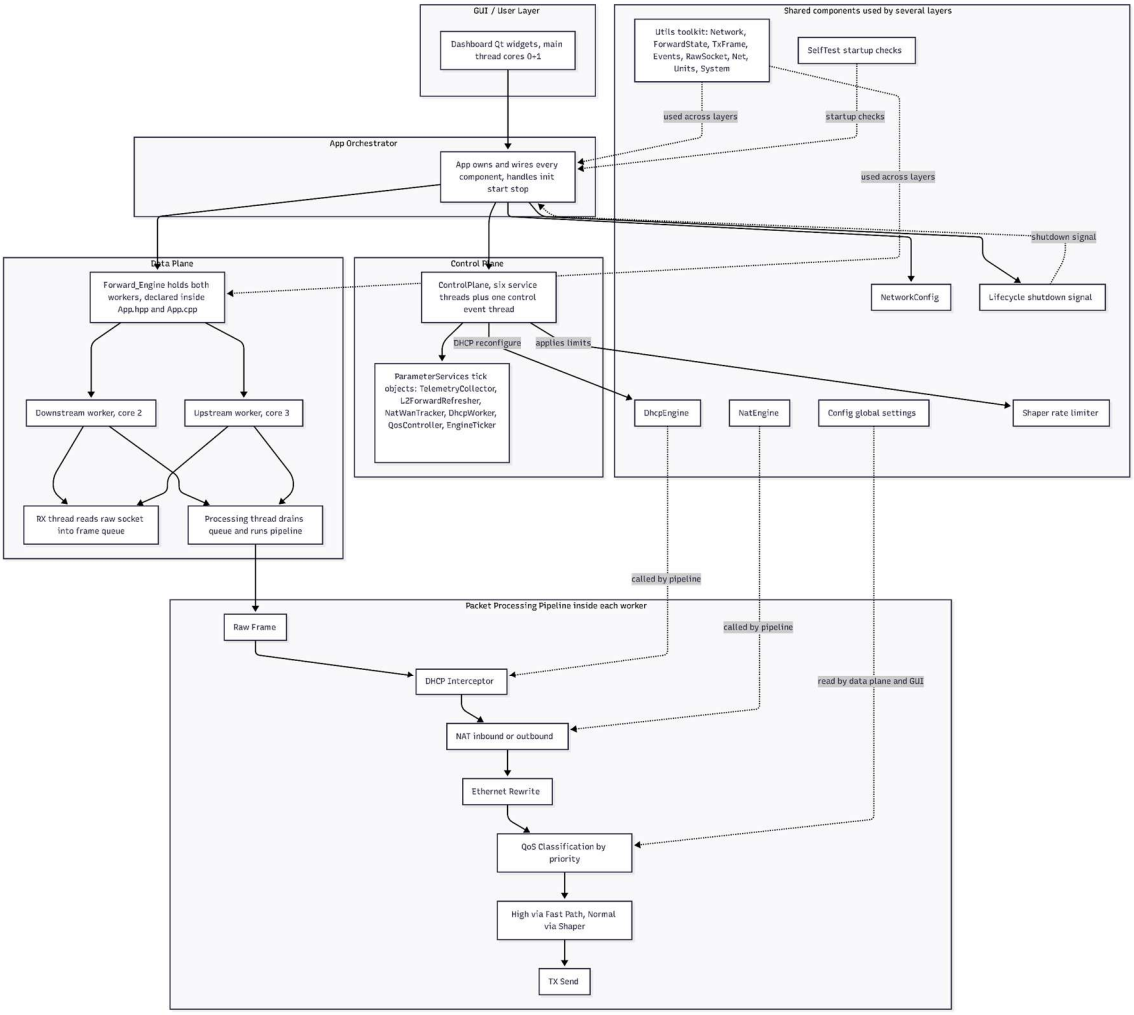


Figure6: Detailed UML Structure (source: plotted by author by Mermaid https://mermaid.ai/)

This diagram presents the overall architecture of our High-Performance Gaming Traffic Prioritizer. The system is divided into several main parts: the GUI/User Layer, App Orchestrator, Data Plane, Control Plane, and shared components. The GUI provides users with a dashboard to monitor the system. The App Orchestrator acts as the central coordinator, responsible for initialising, connecting, starting, and stopping the different components.

The Data Plane is responsible for high-speed packet processing. It contains separate upstream and downstream worker threads, assigned to different CPU cores. Within each worker, the RX thread receives packets through raw sockets and places them into a frame queue, while the processing thread consumes the frames and runs the packet-processing pipeline. This separation allows packet processing to operate independently. It uses an SPSC ring buffer for efficient communication between the two threads, reducing unnecessary synchronisation overhead.

The Control Plane is responsible for managing the system rather than processing every packet directly. The main packet-processing pipeline starts with a raw Ethernet frame. The packet first passes through the DHCP interceptor, followed by NAT processing for inbound or

outbound traffic and then Ethernet header can be rewriting. After that, the packet reaches the QoS classification stage, where traffic is assigned a priority. High-priority traffic can use the fast path, allowing latency-sensitive traffic such as gaming packets to be forwarded quickly. Finally, the processed packet reaches the TX Send stage and it is transmitted to the network.

Overall, the architecture is designed to separate packet processing, system control, and user interaction, while using multithreading mechanisms to maintain high throughput.